GalaxyNet

Classificador Morfológico de Galáxias

Projeto Prático de Deep Learning Aplicado à Astrofísica

Prof. André Ribeiro — UESC

4 de março de 2026

Conteúdo

1 Introdução e Objetivos **3**
1.1 Objetivos de Aprendizagem . . . 3
1.2 Duração e Formato . . . 3
1.3 Motivação Científica: A Morfologia das Galáxias . . . 3

2 Bancos de Dados Utilizados **4**
2.1 SDSS DR17 (Sloan Digital Sky Survey Data Release 17) . . . 4
2.2 Galaxy Zoo 2 . . . 5
2.3 Imagens FITS do SDSS . . . 7

3 Bibliotecas Python Necessárias **10**
3.1 Tabela de Bibliotecas Essenciais . . . 10
3.2 Script de Instalação . . . 10
3.3 Verificação do Ambiente . . . 11

4 Estrutura do Projeto **12**

5 Fase 1 — Exploração e Pré-processamento **13**
5.1 Análise Exploratória de Dados (EDA) . . . 13
5.2 Feature Engineering e Pré-processamento Tabular . . . 14
5.3 Pré-processamento de Imagens . . . 15

6 Fase 2 — Modelo Tabular (MLP) **17**

7 Fase 3 — Modelo CNN com Imagens **20**

8 Fase 4 — Modelo Híbrido **23**

9 Avaliação Científica **25**

10 Dicas Gerais e Boas Práticas **28**
10.1 Reprodutibilidade . . . 28
10.2 Depuração de Erros Comuns . . . 28
10.3 Recursos de Apoio . . . 29

11 Critérios de Avaliação **30**

12 Cronograma Sugerido **31**

A Apêndice A: Template de Relatório Final **32**

B Apêndice B: Checklist Final do Projeto **32**

1 Introdução e Objetivos

Este guia prático foi desenvolvido para estudantes que desejam aplicar seus conhecimentos em Deep Learning a um problema bastante conhecido da astrofísica: a classificação morfológica de galáxias.

Neste projeto, vocês construirão um pipeline completo, desde a aquisição de dados brutos de grandes surveys astronômicos até o treinamento e avaliação de modelos de Deep Learning. O objetivo é aprender "fazendo".

1.1 Objetivos de Aprendizagem

Ao final deste projeto, vocês serão capazes de:

- Acessar e manipular grandes bancos de dados astronômicos (SDSS, Galaxy Zoo).
- Realizar pré-processamento e engenharia de características (feature engineering) para dados fotométricos e imagens de galáxias.
- Implementar e treinar Redes Neurais Artificiais (MLP) e Redes Neurais Convolucionais (CNN) usando TensorFlow/Keras.
- Desenvolver modelos híbridos que combinam diferentes tipos de dados.
- Avaliar modelos usando métricas de performance padrão e métricas específicas da astrofísica.
- Interpretar os resultados de modelos de Deep Learning e discutir suas implicações científicas.

1.2 Duração e Formato

Este projeto está estruturado para ser desenvolvido ao longo de **6 semanas**.

1.3 Motivação Científica: A Morfologia das Galáxias

A morfologia das galáxias — sua forma e estrutura — é um indicador de sua história evolutiva. A sequência de Hubble, que classifica galáxias em elípticas, espirais e irregulares, tem sido a base para entender como as galáxias se formam e evoluem. Com o advento de grandes surveys que observam milhões de galáxias, a classificação manual tornou-se inviável. O Deep Learning surge como uma ferramenta poderosa para automatizar essa tarefa, permitindo descobertas em escalas nunca antes imaginadas.

2 Bancos de Dados Utilizados

Para este projeto, utilizaremos dados reais de dois dos maiores e mais influentes projetos de astronomia.

2.1 SDSS DR17 (Sloan Digital Sky Survey Data Release 17)

O SDSS é um dos mais ambiciosos e bem-sucedidos surveys astronômicos da história, mapeando milhões de galáxias, quasares e estrelas. Ele fornece dados fotométricos (magnitudes em diferentes bandas) e espectroscópicos (redshift, dispersão de velocidades).

Acesso ao SDSS

Para acessar o SDSS pela primeira vez, acesse https://skyserver.sdss.org/dr17/SearchTools/sql e use a ferramenta SQL Search para testar suas queries antes de rodar via Python. Isso ajuda a depurar e entender os dados.

Abaixo, um exemplo de código Python para baixar dados de galáxias do SDSS usando a biblioteca ‘astroquery‘.

Listing 1: Download de Dados Fotométricos e Espectroscópicos do SDSS

```python
import pandas as pd
from astroquery.sdss import SDSS
from astropy.coordinates import SkyCoord
import astropy.units as u

def download_sdss_galaxy_data(ra_center, dec_center, radius_deg, max_records=5000):
    """
    Downloads photometric and spectroscopic data for galaxies from SDSS DR17.

    Args:
        ra_center (float): Right Ascension of the center of the search area (degrees).
        dec_center (float): Declination of the center of the search area (degrees).
        radius_deg (float): Radius of the search area (degrees).
        max_records (int): Maximum number of records to retrieve.

    Returns:
        pd.DataFrame: DataFrame containing the downloaded galaxy data.
    """
    # SQL query to select relevant galaxy properties
    query = f"""
    SELECT TOP {max_records}
        p.objid, p.ra, p.dec,
        p.u, p.g, p.r, p.i, p.z,
        p.petroRad_r, p.petroR50_r, p.petroR90_r,
        p.deVAB_r, p.expAB_r,
        p.lnLDeV_r, p.lnLExp_r, p.lnLStar_r,
        p.fracDeV_r,
        s.z AS redshift, s.zErr,
        s.velDisp, s.velDispErr
    FROM PhotoObj AS p
    JOIN SpecObj AS s ON s.bestobjid = p.objid
```

```
    WHERE
        p.clean = 1 AND  -- Ensure good quality photometric data
        p.r BETWEEN 14.0 AND 19.5 AND -- Filter by r-band magnitude
        s.z BETWEEN 0.02 AND 0.25 AND -- Filter by redshift range
        s.zWarning = 0 AND -- Ensure good quality redshift measurement
        dbo.fDistanceArcMinEq(p.ra, p.dec, {ra_center}, {dec_center}) <
            {radius_deg*60}
    """

    # Execute the query
    result = SDSS.query_sql(query)

    if result is not None:
        df = result.to_pandas()
        print(f"Downloaded {len(df)} records from SDSS.")
        return df
    else:
        print("No results found for the given query.")
        return pd.DataFrame()

# Example usage: Download 5000 galaxies around a specific sky coordinate
# sdss_data = download_sdss_galaxy_data(ra_center=180.0, dec_center=0.0,
    radius_deg=2.0, max_records=5000)
# print(sdss_data.head())
```

Tempo de Execução da Query

A query SQL para o SDSS pode levar vários minutos para ser executada, dependendo do número de registros e da carga do servidor. Para os primeiros testes, é altamente recomendável usar um ‘TOP‘ menor (ex: ‘TOP 1000‘) ou um ‘radius deg‘ menor para acelerar o processo.

2.2 Galaxy Zoo 2

O Galaxy Zoo é um projeto de ciência cidadã onde milhares de voluntários classificaram a morfologia de galáxias. O Galaxy Zoo 2 (GZ2) é uma extensão que fornece classificações detalhadas para mais de 240.000 galáxias do SDSS. Usaremos essas classificações como "verdade fundamental"(ground truth) para treinar nossos modelos.

Download do Galaxy Zoo 2

O dataset completo do Galaxy Zoo 2 pode ser baixado em https://zenodo.org/record/3565489. Procure pelo arquivo ‘gz2 hart16.csv‘.

Listing 2: Carregamento e Filtragem de Dados do Galaxy Zoo 2

```
import pandas as pd

def load_and_filter_gz2(file_path="gz2_hart16.csv"):
    """
    Loads the Galaxy Zoo 2 dataset and filters relevant columns for
        classification.

    Args:
        file_path (str): Path to the gz2_hart16.csv file.
```

```python

    Returns:
        pd.DataFrame: Filtered DataFrame with GZ2 classifications.
    """
    # Columns to keep from the GZ2 dataset
    gz2_cols = [
        'dr8objid',  # Object ID for cross-matching with SDSS
        't01_smooth_or_features_a01_smooth_fraction',
        't01_smooth_or_features_a02_features_or_disk_fraction',
        't01_smooth_or_features_a03_star_or_artifact_fraction',
        't03_bar_a06_bar_fraction',
        't03_bar_a07_no_bar_fraction',
        't04_spiral_a08_spiral_fraction',
        't04_spiral_a09_no_spiral_fraction',
        't01_smooth_or_features_total_weight' # Total votes for confidence
    ]

    gz2_df = pd.read_csv(file_path, usecols=gz2_cols)

    # Rename 'dr8objid' to 'objid' for easier merging with SDSS data
    gz2_df.rename(columns={'dr8objid': 'objid'}, inplace=True)

    print(f"Loaded {len(gz2_df)} records from GZ2.")
    return gz2_df

def assign_morphological_class(row, min_vote_fraction=0.8, min_total_votes=20):
    """
    Assigns a morphological class based on GZ2 vote fractions.

    Args:
        row (pd.Series): A row from the GZ2 DataFrame.
        min_vote_fraction (float): Minimum fraction of votes for a class to be assigned.
        min_total_votes (int): Minimum total votes for a reliable classification.

    Returns:
        str: Assigned class ('Elliptical', 'Spiral', 'Lenticular', 'Irregular', 'Uncertain').
    """
    if row['t01_smooth_or_features_total_weight'] < min_total_votes:
        return 'Uncertain'

    if row['t01_smooth_or_features_a01_smooth_fraction'] >= min_vote_fraction:
        # Smooth galaxies: check for disk features to distinguish Elliptical from Lenticular
        if row['t01_smooth_or_features_a02_features_or_disk_fraction'] >= min_vote_fraction:
            return 'Lenticular' # Smooth with disk features
        else:
            return 'Elliptical' # Purely smooth
    elif row['t01_smooth_or_features_a02_features_or_disk_fraction'] >= min_vote_fraction:
        # Galaxies with features/disk: check for spiral arms
        if row['t04_spiral_a08_spiral_fraction'] >= min_vote_fraction:
```

```
            return 'Spiral'
        else:
            return 'Irregular' # Disk but no clear spiral arms, or other irregular features

    return 'Uncertain' # Default if no clear class is assigned

# Example usage:
# gz2_data = load_and_filter_gz2()
# gz2_data['morph_class'] = gz2_data.apply(assign_morphological_class, axis=1)
# print(gz2_data['morph_class'].value_counts())
```

Classes Morfológicas e Confiança

A coluna ‘gz2class‘ (ou ‘morph-class‘ após a atribuição) será a variável alvo para a classificação. É importante usar um limiar de confiança (ex: ‘min-vote-fraction=0.8‘ e ‘min-total-votes=20‘) para garantir que as classificações sejam robustas e não baseadas em poucos votos ou votos divididos. Galáxias com classificações incertas devem ser removidas do conjunto de treinamento.

2.3 Imagens FITS do SDSS

Para o treinamento de CNNs, precisaremos das imagens reais das galáxias. O SDSS fornece essas imagens no formato FITS (Flexible Image Transport System), um padrão na astronomia.

Listing 3: Download de Imagens FITS do SDSS

```
import numpy as np
from astroquery.sdss import SDSS
from astropy.coordinates import SkyCoord
import astropy.units as u
from astropy.io import fits
import matplotlib.pyplot as plt

def download_galaxy_image_cutout(ra, dec, objid, size_pixels=64, band_list=['g', 'r', 'i']):
    """
    Downloads a cutout image for a galaxy from SDSS in specified bands.

    Args:
        ra (float): Right Ascension of the galaxy (degrees).
        dec (float): Declination of the galaxy (degrees).
        objid (int): SDSS object ID.
        size_pixels (int): Size of the square cutout image in pixels (e.g., 64 for 64x64).
        band_list (list): List of photometric bands to download (e.g., ['g', 'r', 'i']).

    Returns:
        np.ndarray: A 3D NumPy array (size_pixels, size_pixels, len(band_list))
                    containing the image data, or None if download fails.
    """
```

```python
    pos = SkyCoord(ra=ra*u.deg, dec=dec*u.deg, frame='icrs')

    try:
        # Get images for the specified bands
        images = SDSS.get_images(
            coordinates=pos,
            radius=size_pixels * 0.396 * u.arcsec, # 0.396 arcsec/pixel for SDSS
            band=band_list,
            data_release=17,
            cutout_size=size_pixels*u.pixel
        )

        if not images:
            print(f"Warning: No images found for objid {objid} at RA={ra}, Dec={dec}")
            return None

        # Stack images from different bands into a single array
        channels = []
        for img_hdu in images:
            # img_hdu is a list of HDU objects, usually the first one contains the image data
            data = img_hdu[0].data
            if data.shape[0] != size_pixels or data.shape[1] != size_pixels:
                print(f"Warning: Image cutout for objid {objid} has unexpected size {data.shape}. Expected {size_pixels}x{size_pixels}.")
                # Attempt to resize if possible, or return None
                # For simplicity, we'll return None if size is not as expected
                return None
            channels.append(data)

        # Ensure all channels were successfully retrieved
        if len(channels) == len(band_list):
            return np.stack(channels, axis=-1) # Stack along the last axis (channels)
        else:
            print(f"Warning: Missing bands for objid {objid}. Expected {len(band_list)}, got {len(channels)}.")
            return None

    except Exception as e:
        print(f"Error downloading image for objid {objid}: {e}")
        return None

# Example usage:
# ra_sample, dec_sample, objid_sample = 180.0, 0.0, 1237654879999999999 # Replace with actual values
# galaxy_image = download_galaxy_image_cutout(ra_sample, dec_sample, objid_sample)
# if galaxy_image is not None:
#     print(f"Image shape: {galaxy_image.shape}") # Should be (64, 64, 3)
#     # You can then save this image or process it further
```

Download de Imagens em Lotes

Baixar imagens uma a uma pode ser muito lento. Recomenda-se implementar um loop com uma barra de progresso (‘tqdm‘) e salvar as imagens em disco (ex: formato ‘.npy‘ ou ‘.png‘) para evitar downloads repetidos. Comece com uma amostra pequena (ex: 500 imagens) para testar seu pipeline antes de tentar baixar todas as imagens.

3 Bibliotecas Python Necessárias

Para este projeto, utilizaremos diversas bibliotecas Python, cada uma com sua finalidade específica.

3.1 Tabela de Bibliotecas Essenciais

Tabela 1: Bibliotecas Python Essenciais para o Projeto GalaxyNet

Biblioteca	Versão Mínima	Finalidade	Instalação
`tensorflow`	2.8.0	Construção e treino de redes neurais	`pip install tensorflow`
`numpy`	1.21.0	Operações numéricas e arrays	`pip install numpy`
`pandas`	1.4.0	Manipulação de dados tabulares	`pip install pandas`
`matplotlib`	3.5.0	Visualização de dados e resultados	`pip install matplotlib`
`seaborn`	0.11.0	Visualizações estatísticas	`pip install seaborn`
`scikit-learn`	1.1.0	Pré-processamento e métricas	`pip install scikit-learn`
`astropy`	5.0.0	Leitura de arquivos FITS e unidades	`pip install astropy`
`astroquery`	0.4.6	Acesso a catálogos online (SDSS)	`pip install astroquery`
`photutils`	1.4.0	Fotometria e detecção de fontes	`pip install photutils`
`opencv-python`	4.5.0	Processamento de imagens	`pip install opencv-python`
`scipy`	1.8.0	Funções científicas e estatísticas	`pip install scipy`
`tqdm`	4.64.0	Barras de progresso	`pip install tqdm`
`imbalanced-learn`	0.9.0	Balanceamento de classes	`pip install imbalanced-learn`

3.2 Script de Instalação

É uma boa prática criar um arquivo ‘requirements.txt‘ para gerenciar as dependências do seu projeto.

Listing 4: Script de Instalação de Dependências

```
#!/bin/bash

# Create a virtual environment
python -m venv galaxy_env
source galaxy_env/bin/activate # For Linux/macOS
# For Windows, use: .\galaxy_env\Scripts\activate

# Upgrade pip
pip install --upgrade pip

# Create requirements.txt (if not already created)
echo "tensorflow>=2.8.0" > requirements.txt
echo "numpy>=1.21.0" >> requirements.txt
echo "pandas>=1.4.0" >> requirements.txt
echo "matplotlib>=3.5.0" >> requirements.txt
echo "seaborn>=0.11.0" >> requirements.txt
echo "scikit-learn>=1.1.0" >> requirements.txt
echo "astropy>=5.0.0" >> requirements.txt
echo "astroquery>=0.4.6" >> requirements.txt
echo "photutils>=1.4.0" >> requirements.txt
echo "opencv-python>=4.5.0" >> requirements.txt
echo "scipy>=1.8.0" >> requirements.txt
```

```
echo "tqdm>=4.64.0" >> requirements.txt
echo "imbalanced-learn>=0.9.0" >> requirements.txt

# Install all dependencies
pip install -r requirements.txt

echo "All dependencies installed successfully in 'galaxy_env' virtual
    environment."
echo "To deactivate the environment, type 'deactivate'."
```

3.3 Verificação do Ambiente

Após a instalação, verifique se todas as bibliotecas foram instaladas corretamente.

Listing 5: Verificação da Instalação das Bibliotecas

```
import tensorflow as tf
import numpy as np
import pandas as pd
import matplotlib as mpl
import seaborn as sns
import sklearn
import astropy
import astroquery
import photutils
import cv2
import scipy
import tqdm
import imblearn

print(f"TensorFlow version: {tf.__version__}")
print(f"NumPy version: {np.__version__}")
print(f"Pandas version: {pd.__version__}")
print(f"Matplotlib version: {mpl.__version__}")
print(f"Seaborn version: {sns.__version__}")
print(f"Scikit-learn version: {sklearn.__version__}")
print(f"Astropy version: {astropy.__version__}")
print(f"Astroquery version: {astroquery.__version__}")
print(f"Photutils version: {photutils.__version__}")
print(f"OpenCV version: {cv2.__version__}")
print(f"SciPy version: {scipy.__version__}")
print(f"Tqdm version: {tqdm.__version__}")
print(f"Imbalanced-learn version: {imblearn.__version__}")

print("
All required libraries are installed and ready!")
```

4 Estrutura do Projeto

Uma boa organização de projeto é fundamental para a clareza, manutenção e colaboração. Sugerimos a seguinte estrutura de diretórios:

Listing 6: Estrutura de Diretórios Recomendada

```
galaxy_classification/
|-- data/
|   |-- raw/                # Raw data (e.g., gz2_hart16.csv, initial SDSS data)
|   |-- processed/          # Processed data (e.g., merged catalog, preprocessed images)
|   `-- images/             # Downloaded FITS images or processed image arrays
|-- notebooks/              # Jupyter notebooks for exploration and development
|   |-- 01_eda.ipynb
|   |-- 02_preprocessing.ipynb
|   |-- 03_mlp_model.ipynb
|   |-- 04_cnn_model.ipynb
|   `-- 05_hybrid_model.ipynb
|-- src/                    # Source code for reusable functions and classes
|   |-- data_loader.py      # Functions to load and download data
|   |-- preprocessing.py    # Functions for data cleaning and feature engineering
|   |-- models.py           # Keras model definitions
|   |-- evaluation.py       # Functions for model evaluation and metrics
|   `-- visualization.py    # Functions for plotting results
|-- models/                 # Saved trained models (e.g., .h5 or SavedModel format)
|-- reports/                # Final reports, figures, and presentation materials
|-- requirements.txt        # List of project dependencies
`-- README.md               # Project documentation and instructions
```

Controle de Versão com Git

Utilizem Git para controle de versão do projeto. Façam commits frequentes e significativos. Isso permite rastrear mudanças, colaborar de forma eficiente e reverter para versões anteriores, se necessário. Recomenda-se fazer um commit após cada grande etapa do projeto (ex: "Dados baixados", "Pré-processamento concluído", "MLP treinado").

5 Fase 1 — Exploração e Pré-processamento

Esta fase é importante para entender os dados e prepará-los para o treinamento dos modelos.

5.1 Análise Exploratória de Dados (EDA)

A EDA permite que vocês se familiarizem com as características dos dados, identifiquem padrões, anomalias e problemas potenciais.

Listing 7: Exemplo de EDA Básica para Dados SDSS

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Assuming sdss_data and gz2_data are already loaded and merged
# merged_df = pd.merge(sdss_data, gz2_data, on='objid', how='inner')

# Basic statistics
# print(merged_df.describe())

# Check for missing values
# print(merged_df.isnull().sum())

# Distribution of redshift
plt.figure(figsize=(8, 5))
sns.histplot(merged_df['redshift'], bins=50, kde=True)
plt.title('Distribution of Redshift')
plt.xlabel('Redshift (z)')
plt.ylabel('Count')
plt.show()

# Distribution of morphological classes
plt.figure(figsize=(8, 5))
sns.countplot(x='morph_class', data=merged_df)
plt.title('Distribution of Morphological Classes')
plt.xlabel('Galaxy Class')
plt.ylabel('Count')
plt.show()

# Color-magnitude diagram (example: g-r vs r)
plt.figure(figsize=(10, 7))
sns.scatterplot(x=merged_df['g'] - merged_df['r'], y=merged_df['r'],
                hue=merged_df['morph_class'], alpha=0.5, s=10)
plt.title('Color-Magnitude Diagram (g-r vs r)')
plt.xlabel('g - r color')
plt.ylabel('r magnitude')
plt.legend(title='Galaxy Class')
plt.gca().invert_yaxis() # Magnitudes are usually inverted
plt.show()
```

Verificação da EDA

Antes de prosseguir, certifiquem-se de que:

1. Os dados do SDSS e Galaxy Zoo foram carregados e mesclados corretamente.

2. Não há valores nulos significativos nas colunas que serão utilizadas.
3. A distribuição das classes morfológicas foi verificada (atenção ao desbalanceamento).
4. Os índices de cor e outras características derivadas foram calculados.
5. Pelo menos 3 gráficos relevantes foram gerados para entender os dados.

5.2 Feature Engineering e Pré-processamento Tabular

Criar novas características a partir das existentes e normalizar os dados é essencial para o bom desempenho dos modelos.

Listing 8: Feature Engineering e Normalização de Dados Tabulares

```python
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

def preprocess_tabular_data(df):
    """
    Performs feature engineering and scaling for tabular data.

    Args:
        df (pd.DataFrame): Merged DataFrame with SDSS and GZ2 data.

    Returns:
        tuple: X (features), y (labels), scaler (fitted scaler object).
    """
    # Calculate color indices
    df['u_g'] = df['u'] - df['g']
    df['g_r'] = df['g'] - df['r']
    df['r_i'] = df['r'] - df['i']
    df['i_z'] = df['i'] - df['z']

    # Calculate concentration index
    # Ensure petroR50_r is not zero to avoid division by zero
    df['concentration'] = df['petroR90_r'] / df['petroR50_r'].replace(0,
        np.nan)

    # Select features for the model
    features = [
        'u', 'g', 'r', 'i', 'z',
        'u_g', 'g_r', 'r_i', 'i_z',
        'redshift', 'velDisp',
        'concentration', 'deVAB_r', 'expAB_r', 'fracDeV_r'
    ]

    # Drop rows with any NaN values that might have resulted from
       feature engineering
    df_cleaned = df.dropna(subset=features + ['morph_class'])

    X = df_cleaned[features]
    y = df_cleaned['morph_class']

    # Scale numerical features
    scaler = StandardScaler()
```

```
    X_scaled = scaler.fit_transform(X)

    return X_scaled, y, scaler, df_cleaned['objid'] # Return objid for image matching

# Example usage:
# X_tabular, y_labels, scaler, objids_cleaned = preprocess_tabular_data(merged_df)
# print(X_tabular.shape, y_labels.shape)
```

Normalização Arcsinh

Para dados astronômicos, especialmente magnitudes e fluxos, a normalização ‘arcsinh‘ é frequentemente preferida à ‘StandardScaler‘ ou ‘MinMaxScaler‘ porque lida melhor com a natureza logarítmica das magnitudes e com a presença de valores próximos de zero. No entanto, para este projeto, ‘StandardScaler‘ é um bom ponto de partida.

5.3 Pré-processamento de Imagens

As imagens FITS precisam ser lidas, normalizadas e redimensionadas para serem usadas em CNNs.

Listing 9: Pré-processamento de Imagens FITS

```
import numpy as np
from astropy.io import fits
import cv2 # OpenCV for image processing
from skimage import exposure # For arcsinh stretch

def preprocess_galaxy_image(image_data, target_size=(64, 64), stretch_factor=1000):
    """
    Preprocesses a 3D galaxy image array (H, W, C) for CNN input.

    Args:
        image_data (np.ndarray): 3D NumPy array of the image (H, W, C).
        target_size (tuple): Desired (height, width) for the output image.
        stretch_factor (float): Factor for arcsinh stretch normalization.

    Returns:
        np.ndarray: Preprocessed image array (target_size[0], target_size[1], C),
                    normalized to [0, 1].
    """
    if image_data is None:
        return None

    # Apply arcsinh stretch normalization to each channel independently
    # This helps to bring out faint features without saturating bright ones
    normalized_channels = []
    for i in range(image_data.shape[-1]): # Iterate over channels
        channel = image_data[:, :, i]
        # Apply arcsinh stretch
        stretched_channel = exposure.arcsinh(channel / stretch_factor)
```

```
        # Scale to [0, 1]
        stretched_channel = (stretched_channel - stretched_channel.min()) / \
                            (stretched_channel.max() - stretched_channel.min())
        normalized_channels.append(stretched_channel)

    # Stack back into a 3D array
    normalized_image = np.stack(normalized_channels, axis=-1)

    # Resize image to target_size
    # OpenCV expects (width, height) for resize
    resized_image = cv2.resize(normalized_image, target_size,
                               interpolation=cv2.INTER_AREA)

    return resized_image.astype(np.float32) # Ensure float32 for TensorFlow

# Example usage:
# Assuming 'galaxy_image' is a (H, W, C) array from download_galaxy_image_cutout
# preprocessed_img = preprocess_galaxy_image(galaxy_image)
# if preprocessed_img is not None:
#     print(f"Preprocessed image shape: {preprocessed_img.shape}") # Should be (64, 64, 3)
#     print(f"Min value: {preprocessed_img.min()}, Max value: {preprocessed_img.max()}") # Should be between 0 and 1
```

Normalização de Imagens

É fundamental que as imagens de entrada para a CNN estejam normalizadas, geralmente no intervalo $[0, 1]$ ou $[-1, 1]$. A função ‘preprocess-galaxy-image‘ já faz isso. Imagens não normalizadas podem levar a problemas de treinamento (gradientes explosivos/desaparecendo) e convergência lenta.

6 Fase 2 — Modelo Tabular (MLP)

Nesta fase, vocês construirão e treinarão uma Rede Neural Multi-Camadas (MLP) usando apenas as características fotométricas e espectroscópicas das galáxias.

Listing 10: Implementação e Treinamento de um MLP para Classificação de Galáxias

```python
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.utils import to_categorical
import numpy as np

def create_mlp_model(input_shape, num_classes):
    """
    Creates a Multi-Layer Perceptron (MLP) model for galaxy
        classification.

    Args:
        input_shape (int): Number of input features.
        num_classes (int): Number of output classes.

    Returns:
        keras.Model: Compiled Keras MLP model.
    """
    model = keras.Sequential([
        layers.Input(shape=(input_shape,)),
        layers.Dense(256, activation='relu'),
        layers.BatchNormalization(),
        layers.Dropout(0.3),
        layers.Dense(128, activation='relu'),
        layers.BatchNormalization(),
        layers.Dropout(0.3),
        layers.Dense(64, activation='relu'),
        layers.BatchNormalization(),
        layers.Dropout(0.2),
        layers.Dense(num_classes, activation='softmax')
    ])

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=0.001),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )
    return model

def train_mlp_model(X_tabular, y_labels, objids_cleaned):
    """
    Prepares data, trains, and evaluates the MLP model.

    Args:
        X_tabular (np.ndarray): Scaled tabular features.
        y_labels (pd.Series): Morphological class labels.
        objids_cleaned (pd.Series): Object IDs corresponding to
            X_tabular and y_labels.

    Returns:
```

```
        keras.Model: Trained MLP model.
        tuple: (X_test, y_test_encoded, objids_test) for later evaluation.
    """
    # Encode string labels to integers
    label_encoder = LabelEncoder()
    integer_encoded = label_encoder.fit_transform(y_labels)
    y_one_hot = to_categorical(integer_encoded)

    # Split data into training, validation, and test sets
    X_train, X_temp, y_train, y_temp, objids_train, objids_temp = train_test_split(
        X_tabular, y_one_hot, objids_cleaned, test_size=0.3, random_state=42, stratify=y_one_hot
    )
    X_val, X_test, y_val, y_test, objids_val, objids_test = train_test_split(
        X_temp, y_temp, objids_temp, test_size=0.5, random_state=42, stratify=y_temp
    )

    print(f"Train samples: {len(X_train)}, Val samples: {len(X_val)}, Test samples: {len(X_test)}")

    # Create and compile the model
    input_shape = X_train.shape[1]
    num_classes = y_train.shape[1]
    mlp_model = create_mlp_model(input_shape, num_classes)

    # Define callbacks
    callbacks = [
        keras.callbacks.EarlyStopping(
            monitor='val_loss', patience=10, restore_best_weights=True, verbose=1
        ),
        keras.callbacks.ReduceLROnPlateau(
            monitor='val_loss', factor=0.5, patience=5, min_lr=1e-7, verbose=1
        )
    ]

    # Train the model
    history = mlp_model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=100, # Max epochs, EarlyStopping will stop it
        batch_size=32,
        callbacks=callbacks,
        verbose=1
    )

    # Evaluate on test set
    loss, accuracy = mlp_model.evaluate(X_test, y_test, verbose=0)
    print(f"MLP Test Loss: {loss:.4f}, Test Accuracy: {accuracy:.4f}")

    return mlp_model, (X_test, y_test, objids_test, label_encoder)

# Example usage:
```

```
# mlp_model, (X_test_mlp, y_test_mlp, objids_test_mlp, label_encoder_mlp
    ) = train_mlp_model(X_tabular, y_labels, objids_cleaned)
```

Batch Normalization e Dropout

A ‘BatchNormalization‘ ajuda a estabilizar o treinamento de redes neurais profundas, enquanto o ‘Dropout‘ atua como uma técnica de regularização, prevenindo o overfitting. A ordem recomendada é ‘Dense -> BatchNormalization -> ReLU -> Dropout‘.

MLP Treinado

Antes de avançar para a próxima fase, certifiquem-se de que:

1. O modelo MLP foi treinado e avaliado.
2. A acurácia no conjunto de validação é superior a 80% (ou um valor razoável para o problema).
3. As curvas de perda e acurácia (treinamento vs. validação) foram plotadas para verificar overfitting/underfitting.
4. O modelo foi salvo em disco (‘mlp-model.save(’models/mlp-galaxy-classifier.h5’)‘).

7 Fase 3 — Modelo CNN com Imagens

Agora, vocês construirão uma Rede Neural Convolucional (CNN) para classificar galáxias usando suas imagens.

Listing 11: Implementação e Treinamento de uma CNN para Classificação de Galáxias

```python
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import numpy as np
import os

def create_cnn_model(input_shape=(64, 64, 3), num_classes=5):
    """
    Creates a Convolutional Neural Network (CNN) model for galaxy image classification.

    Args:
        input_shape (tuple): Shape of the input images (height, width, channels).
        num_classes (int): Number of output classes.

    Returns:
        keras.Model: Compiled Keras CNN model.
    """
    model = keras.Sequential([
        layers.Input(shape=input_shape),

        # Block 1
        layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
        layers.BatchNormalization(),
        layers.MaxPooling2D((2, 2)),
        layers.Dropout(0.25),

        # Block 2
        layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
        layers.BatchNormalization(),
        layers.MaxPooling2D((2, 2)),
        layers.Dropout(0.25),

        # Block 3
        layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
        layers.BatchNormalization(),
        layers.MaxPooling2D((2, 2)),
        layers.Dropout(0.25),

        # Block 4
        layers.Conv2D(256, (3, 3), activation='relu', padding='same'),
        layers.BatchNormalization(),
        layers.MaxPooling2D((2, 2)),
        layers.Dropout(0.25),

        # Classifier head
        layers.Flatten(),
        layers.Dense(512, activation='relu'),
        layers.BatchNormalization(),
        layers.Dropout(0.5),
```

```
        layers.Dense(num_classes, activation='softmax')
    ])

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=0.001),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )
    return model

def train_cnn_model(X_images, y_one_hot, objids_cleaned):
    """
    Prepares image data, trains, and evaluates the CNN model.

    Args:
        X_images (np.ndarray): Preprocessed galaxy images.
        y_one_hot (np.ndarray): One-hot encoded labels.
        objids_cleaned (pd.Series): Object IDs corresponding to images and labels.

    Returns:
        keras.Model: Trained CNN model.
        tuple: (X_test, y_test_encoded, objids_test) for later evaluation.
    """
    # Split data
    X_train, X_temp, y_train, y_temp, objids_train, objids_temp = train_test_split(
        X_images, y_one_hot, objids_cleaned, test_size=0.3, random_state=42, stratify=y_one_hot
    )
    X_val, X_test, y_val, y_test, objids_val, objids_test = train_test_split(
        X_temp, y_temp, objids_temp, test_size=0.5, random_state=42, stratify=y_temp
    )

    # Data augmentation
    datagen = ImageDataGenerator(
        rotation_range=360,      # Galaxies have no preferred orientation
        horizontal_flip=True,
        vertical_flip=True,
        zoom_range=0.2,          # Simulate different distances
        brightness_range=[0.8, 1.2], # Vary brightness
        fill_mode='constant',    # Fill with zeros (dark sky)
        cval=0.0
    )

    # Create and compile the model
    input_shape = X_train.shape[1:]
    num_classes = y_train.shape[1]
    cnn_model = create_cnn_model(input_shape, num_classes)

    # Define callbacks
    callbacks = [
        keras.callbacks.EarlyStopping(
            monitor='val_loss', patience=15, restore_best_weights=True,
```

```
                verbose=1
        ),
        keras.callbacks.ReduceLROnPlateau(
            monitor='val_loss', factor=0.5, patience=7, min_lr=1e-7,
                verbose=1
        )
    ]

    # Train the model using data augmentation
    history = cnn_model.fit(
        datagen.flow(X_train, y_train, batch_size=32),
        validation_data=(X_val, y_val),
        epochs=100,
        callbacks=callbacks,
        verbose=1
    )

    # Evaluate on test set
    loss, accuracy = cnn_model.evaluate(X_test, y_test, verbose=0)
    print(f"CNN Test Loss: {loss:.4f}, Test Accuracy: {accuracy:.4f}")

    return cnn_model, (X_test, y_test, objids_test)

# Example usage:
# cnn_model, (X_test_cnn, y_test_cnn, objids_test_cnn) = train_cnn_model
    (X_images, y_one_hot, objids_cleaned)
```

Data Augmentation para Galáxias

A ‘rotation-range=360‘ é particularmente útil para galáxias, pois sua orientação no céu é aleatória. Rotacionar as imagens em qualquer ângulo ajuda o modelo a aprender características que são invariantes à rotação.

Memória e Imagens

Carregar todas as imagens na memória de uma vez pode ser um problema para datasets grandes. Se vocês enfrentarem erros de memória, considerem usar ‘tf.data.Dataset‘ ou ‘ImageDataGenerator.flow-from-directory‘ com as imagens salvas em diretórios, o que carrega os dados em lotes (batches) conforme necessário.

8 Fase 4 — Modelo Híbrido

Nesta fase, vocês combinarão as vantagens das características tabulares e das imagens, criando um modelo híbrido com duas entradas.

Listing 12: Implementação de um Modelo Híbrido (CNN + MLP)

```python
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np

def create_hybrid_model(image_input_shape, tabular_input_shape, num_classes):
    """
    Creates a hybrid model combining CNN for images and MLP for tabular data.

    Args:
        image_input_shape (tuple): Shape of the input images (height, width, channels).
        tabular_input_shape (int): Number of input tabular features.
        num_classes (int): Number of output classes.

    Returns:
        keras.Model: Compiled Keras hybrid model.
    """
    # --- Image Branch (CNN) ---
    image_input = layers.Input(shape=image_input_shape, name='image_input')
    x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(image_input)
    x = layers.BatchNormalization()(x)
    x = layers.MaxPooling2D((2, 2))(x)
    x = layers.Dropout(0.25)(x)

    x = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(x)
    x = layers.BatchNormalization()(x)
    x = layers.MaxPooling2D((2, 2))(x)
    x = layers.Dropout(0.25)(x)

    x = layers.GlobalAveragePooling2D()(x) # Reduce spatial dimensions
    image_features = layers.Dense(128, activation='relu')(x)
    image_features = layers.BatchNormalization()(image_features)
    image_features = layers.Dropout(0.3)(image_features)

    # --- Tabular Branch (MLP) ---
    tabular_input = layers.Input(shape=(tabular_input_shape,), name='tabular_input')
    y = layers.Dense(64, activation='relu')(tabular_input)
    y = layers.BatchNormalization()(y)
    y = layers.Dropout(0.2)(y)
    tabular_features = layers.Dense(32, activation='relu')(y)
    tabular_features = layers.BatchNormalization()(tabular_features)
    tabular_features = layers.Dropout(0.2)(tabular_features)

    # --- Concatenate and Final Classification Head ---
    combined_features = layers.concatenate([image_features, tabular_features])
```

```python

    z = layers.Dense(128, activation='relu')(combined_features)
    z = layers.BatchNormalization()(z)
    z = layers.Dropout(0.4)(z)
    output = layers.Dense(num_classes, activation='softmax')(z)

    # Create the model with two inputs and one output
    hybrid_model = keras.Model(inputs=[image_input, tabular_input],
        outputs=output)

    hybrid_model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=0.001),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )
    return hybrid_model

# Example usage:
# Assuming X_images, X_tabular, y_one_hot are prepared and aligned by
    objid
# image_input_shape = X_images.shape[1:]
# tabular_input_shape = X_tabular.shape[1]
# num_classes = y_one_hot.shape[1]
# hybrid_model = create_hybrid_model(image_input_shape,
    tabular_input_shape, num_classes)

# To train, you'll need to pass a list of inputs:
# history = hybrid_model.fit(
#     [X_train_images, X_train_tabular], y_train_one_hot,
#     validation_data=([X_val_images, X_val_tabular], y_val_one_hot),
#     epochs=100, batch_size=32, callbacks=callbacks
# )
```

Vantagem do Modelo Híbrido

A combinação de características de imagem (capturadas pela CNN) e características tabulares (como redshift, magnitudes e índices de cor) pode levar a um desempenho superior. A CNN é excelente para extrair padrões espaciais, enquanto as características tabulares fornecem informações contextuais importantes que a imagem sozinha pode não revelar.

9 Avaliação Científica

A avaliação de modelos em astrofísica vai além da acurácia. É importante entender o desempenho do modelo para cada classe e quantificar a incerteza.

Listing 13: Métricas de Avaliação Científica e Matriz de Confusão

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report

def evaluate_galaxy_classifier(model, X_test_data, y_test_one_hot, label_encoder, model_type='MLP'):
    """
    Evaluates the galaxy classification model and prints scientific metrics.

    Args:
        model (keras.Model): The trained Keras model.
        X_test_data (np.ndarray or list): Test features (images, tabular, or both).
        y_test_one_hot (np.ndarray): One-hot encoded true labels.
        label_encoder (LabelEncoder): Fitted LabelEncoder object.
        model_type (str): Type of model ('MLP', 'CNN', 'Hybrid').

    Returns:
        None
    """
    # Get predictions
    y_pred_one_hot = model.predict(X_test_data)
    y_pred_labels = label_encoder.inverse_transform(np.argmax(y_pred_one_hot, axis=1))
    y_true_labels = label_encoder.inverse_transform(np.argmax(y_test_one_hot, axis=1))

    class_names = label_encoder.classes_

    print(f"
--- Evaluation for {model_type} Model ---")

    # Classification Report
    print("
Classification Report:")
    print(classification_report(y_true_labels, y_pred_labels, target_names=class_names))

    # Confusion Matrix
    cm = confusion_matrix(y_true_labels, y_pred_labels, labels=class_names)
    plt.figure(figsize=(8, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=class_names, yticklabels=class_names)
    plt.title(f'Confusion Matrix - {model_type}')
    plt.xlabel('Predicted Label')
    plt.ylabel('True Label')
    plt.show()

```

```python
    # Scientific Metrics (Completeness and Reliability)
    results = []
    for i, cls in enumerate(class_names):
        tp = cm[i, i]
        fp = cm[:, i].sum() - tp # False positives for this class
        fn = cm[i, :].sum() - tp # False negatives for this class

        completeness = tp / (tp + fn) if (tp + fn) > 0 else 0 # Recall
        reliability  = tp / (tp + fp) if (tp + fp) > 0 else 0 # Precision

        results.append({
            'Class': cls,
            'N_True': cm[i, :].sum(),
            'Completeness (Recall)': f'{completeness:.3f}',
            'Reliability (Precision)': f'{reliability:.3f}'
        })

    print("
Scientific Metrics (Completeness and Reliability per Class):")
    print(pd.DataFrame(results).to_string(index=False))

# Example usage:
# evaluate_galaxy_classifier(mlp_model, X_test_mlp, y_test_mlp, label_encoder_mlp, model_type='MLP')
# evaluate_galaxy_classifier(cnn_model, X_test_cnn, y_test_cnn, label_encoder_cnn, model_type='CNN')
# evaluate_galaxy_classifier(hybrid_model, [X_test_images, X_test_tabular], y_test_hybrid, label_encoder_hybrid, model_type='Hybrid')
```

Completeness (Recall) vs. Reliability (Precision)

Em astronomia, ‘Completeness‘ (equivalente ao Recall) mede a fração de objetos de uma classe que foram corretamente identificados pelo modelo. ‘Reliability‘ (equivalente à Precision) mede a fração de objetos que o modelo identificou como sendo de uma classe e que realmente pertencem a essa classe. Usualmente, estas frações são chamadas de completeza e pureza. Ambas são fundamentais para entender o desempenho do modelo em contextos de survey.

Checklist Final de Avaliação

Antes de finalizar o projeto, verifiquem que:

1. Todos os três modelos (MLP, CNN, Híbrido) foram avaliados no conjunto de teste.
2. As matrizes de confusão foram geradas e analisadas para cada modelo.
3. As métricas de Completeness e Reliability foram calculadas por classe.
4. Uma tabela comparativa de desempenho entre os três modelos foi criada.
5. Os resultados foram interpretados em termos astrofísicos (ex: qual classe é mais difícil de classificar?).

6. Pelo menos um exemplo de galáxia mal classificada foi analisado para entender o erro.
7. O modelo final (o de melhor desempenho) foi salvo.
8. O código de avaliação está bem documentado e é fácil de usar.

10 Dicas Gerais e Boas Práticas

10.1 Reprodutibilidade

Para garantir que seus resultados possam ser replicados por outros (e por vocês mesmos no futuro), é essencial controlar a aleatoriedade.

Listing 14: Configuração de Seeds para Reprodutibilidade

```python
import numpy as np
import tensorflow as tf
import random
import os

def set_all_seeds(seed=42):
    """
    Sets seeds for reproducibility across different libraries.

    Args:
        seed (int): The seed value to use.
    """
    os.environ['PYTHONHASHSEED'] = str(seed)
    random.seed(seed)
    np.random.seed(seed)
    tf.random.set_seed(seed)
    # For deterministic operations in TensorFlow (might impact
        performance)
    # tf.config.experimental.enable_op_determinism()
    print(f"All seeds set to {seed} for reproducibility.")

# Call this function at the very beginning of your scripts or notebooks
# set_all_seeds(42)
```

10.2 Depuração de Erros Comuns

Aqui estão alguns erros comuns que os estudantes encontram e como resolvê-los:

- **Shape mismatch entre labels e predictions**: Verifiquem se ‘y-true‘ e ‘y-pred‘ têm o mesmo formato (ex: ambos one-hot encoded ou ambos inteiros). Usem ‘to-categorical‘ para one-hot encoding e ‘np.argmax‘ para converter de volta.

- **NaN na função de perda durante o treinamento**: Isso geralmente indica gradientes explosivos. Tentem reduzir a taxa de aprendizado (‘learning-rate‘), usar ‘BatchNormalization‘, ou aplicar ‘gradient clipping‘.

- **Erro de memória ao carregar imagens**: Se estiverem carregando todas as imagens de uma vez, a memória RAM pode esgotar. Usem ‘ImageDataGenerator‘ ou ‘tf.data.Dataset‘ para carregar as imagens em lotes (batches) sob demanda.

- **Desbalanceamento de classes causando modelo viciado**: Se uma classe tem muito mais exemplos que outras, o modelo pode aprender a prever sempre a classe majoritária. Usem técnicas como ‘class-weight‘ no ‘model.fit‘, oversampling (SMOTE) ou undersampling (RandomUnderSampler) da biblioteca ‘imbalanced-learn‘.

- **Vazamento de dados (data leakage) através da normalização**: Certifiquem-se de que o ‘StandardScaler‘ (ou qualquer outro scaler) é ‘fit‘ apenas nos dados de treinamento e depois ‘transform‘ nos dados de validação e teste. Nunca ‘fit‘ em todo o dataset antes da divisão.

10.3 Recursos de Apoio

- **SDSS SkyServer**: https://skyserver.sdss.org/dr17/. Para explorar dados e testar queries.
- **Galaxy Zoo**: https://www.zooniverse.org/projects/zookeeper/galaxy-zoo/. Para entender o projeto de ciência cidadã.
- **Documentação TensorFlow/Keras**: https://www.tensorflow.org/api_docs/python/tf/keras. Para detalhes sobre camadas, modelos e treinamento.
- **Documentação Astropy**: https://docs.astropy.org/en/stable/. Para manipulação de dados astronômicos.
- **Stack Overflow Astronomy**: https://astronomy.stackexchange.com/. Para dúvidas específicas de astronomia.

11 Critérios de Avaliação

A avaliação do projeto será baseada nos seguintes critérios:

Tabela 2: Critérios de Avaliação do Projeto GalaxyNet

Critério	**Peso**	**Descrição**
Aquisição e limpeza dos dados	15%	Pipeline funcional para download, mesclagem e pré-processamento de dados tabulares e imagens.
Modelo MLP tabular	20%	Implementação correta, treinamento, otimização e análise de resultados do modelo MLP.
Modelo CNN com imagens	25%	Arquitetura CNN adequada, uso de data augmentation, treinamento e análise de resultados.
Modelo híbrido	20%	Integração bem-sucedida dos ramos de imagem e tabular, treinamento e comparação de desempenho.
Relatório final	20%	Qualidade do relatório (formato artigo científico, clareza, discussão, referências).

Originalidade e Compreensão

Este projeto é uma oportunidade para aplicar e aprofundar seus conhecimentos. Códigos prontos encontrados na internet podem ser usados também, desde que se compreenda seu funcionamento. Busquem originalidade nas escolhas de arquitetura, pré-processamento e análise (ou seja, montem o pipeline do jeito que acharem melhor). A compreensão dos conceitos e a capacidade de justificar suas decisões serão valorizadas.

12 Cronograma Sugerido

Este cronograma é uma sugestão para guiar o desenvolvimento do projeto ao longo das 6 semanas.

Tabela 3: Cronograma Sugerido para o Projeto GalaxyNet

Semana	**Tema Principal**	**Entregável**
1	Introdução, EDA e Aquisição de Dados	Notebook de EDA inicial, dados SDSS e GZ2 baixados.
2	Pré-processamento e Feature Engineering	Dados tabulares limpos e normalizados, imagens pré-processadas.
3	Modelo MLP Tabular	MLP treinado, avaliado e salvo. Relatório de performance.
4	Modelo CNN com Imagens	CNN treinada, avaliada e salva. Relatório de performance.
5	Modelo Híbrido e Interpretabilidade	Modelo híbrido treinado, avaliação comparativa, análise de Grad-CA
6	Análise Científica e Relatório Final	Relatório final completo, apresentação preparada.

A Apêndice A: Template de Relatório Final

O relatório final deve seguir o formato de um artigo científico, com as seguintes seções:

- **Resumo (Abstract)**: Breve descrição do projeto, métodos e principais resultados.
- **Introdução**: Contexto científico (morfologia de galáxias), motivação do projeto e objetivos.
- **Dados**: Descrição dos datasets SDSS e Galaxy Zoo 2, variáveis utilizadas e processo de aquisição.
- **Métodos**: Detalhes sobre o pré-processamento, feature engineering, arquiteturas dos modelos (MLP, CNN, Híbrido) e estratégias de treinamento.
- **Resultados**: Apresentação das métricas de avaliação (acurácia, completeness, reliability, matriz de confusão) para cada modelo, comparação entre eles e exemplos de interpretação (Grad-CAM).
- **Discussão**: Análise crítica dos resultados, limitações dos modelos, implicações astrofísicas e sugestões para trabalhos futuros.
- **Conclusão**: Sumário das principais descobertas e contribuições do projeto.
- **Referências**: Lista de todas as fontes citadas (artigos, documentações, etc.).

B Apêndice B: Checklist Final do Projeto

Antes da submissão, verifiquem se todos os itens abaixo foram concluídos:

- ☐ O ambiente Python está configurado e todas as dependências estão instaladas.
- ☐ Os dados do SDSS e Galaxy Zoo 2 foram baixados e mesclados.
- ☐ A EDA foi realizada e documentada.
- ☐ Os dados tabulares foram pré-processados e as características foram escaladas.
- ☐ As imagens FITS foram baixadas e pré-processadas.
- ☐ O modelo MLP foi implementado, treinado e avaliado.
- ☐ O modelo CNN foi implementado, treinado e avaliado (com data augmentation).
- ☐ O modelo híbrido foi implementado, treinado e avaliado.
- ☐ Todos os modelos foram salvos em disco.
- ☐ As métricas de avaliação científica (completeness, reliability) foram calculadas.
- ☐ As matrizes de confusão foram geradas para cada modelo.
- ☐ Uma análise de interpretabilidade (ex: Grad-CAM) foi realizada.

- ☐ O código está organizado, comentado e segue as boas práticas.
- ☐ O relatório final foi escrito no formato de artigo científico.
- ☐ A apresentação oral está preparada.
- ☐ O repositório Git está atualizado com todos os arquivos do projeto.

Bons estudos e boas descobertas!

Prof. André Ribeiro